Predicción del éxito comercial de videojuegos mediante Inteligencia Artificial

Ainhoa Lodeiro López
Curso de especialización de Inteligencia Artificial y Big Data

ÍNDICE

Planteamiento del problema 3
Obtención y tratamiento de datos 4
Muestra de datos: 5
Desarrollo de la solución IA/Big Data 7
Random Forest Regressor 7
Redes Neuronales 11
Comparativa 15
Conclusión 16

Planteamiento del problema

Una de las decisiones más críticas dentro de la industria de los videojuegos es evaluar el potencial comercial de un nuevo título antes de su lanzamiento. Esta evaluación influye directamente en múltiples aspectos clave del desarrollo, como la inversión en marketing, la contratación de doblaje y localización, la elección del motor gráfico, la inclusión de música original o el diseño de la jugabilidad. El desarrollo de un videojuego puede implicar millones de euros en costes, por lo que estimar su probabilidad de éxito es fundamental para garantizar la rentabilidad del proyecto. Por ello, este proyecto se centrará específicamente en el mercado europeo, que representa uno de los sectores más relevantes y diversos de la industria global del videojuego.

Para ello, es necesario realizar estudios comparativos y análisis estadísticos en base a videojuegos similares ya lanzados: su género, plataforma, editor, región de lanzamiento y otros factores que puedan influir en su desempeño comercial. Este tipo de análisis permite a los estudios tomar decisiones informadas, minimizar riesgos económicos y optimizar recursos. Por lo tanto, desarrollar una herramienta basada en Inteligencia Artificial que permita predecir el éxito comercial de un videojuego (a partir de variables clave) resulta no solo útil, sino estratégico en un sector cada vez más competitivo y saturado.

Esta solución podría ser una herramienta de gran valor para estudios de desarrollo de videojuegos, especialmente para pequeños desarrolladores o estudios independientes que disponen de presupuestos limitados y necesitan maximizar el rendimiento de sus proyectos. Contar con una estimación fiable del éxito comercial potencial les permitiría tomar decisiones más estratégicas sobre el uso de herramientas, la inversión en tecnologías más avanzadas, y la planificación del desarrollo. Además, sería útil para diseñar campañas de marketing más efectivas, optimizando recursos y aumentando las probabilidades de éxito en un mercado altamente competitivo, como ya hemos dicho antes.

Para lograrlo, se empleará un conjunto de datos reales de videojuegos más vendidos, con especial atención a las cifras de ventas en Europa, realizando un proceso de limpieza, exploración de relaciones entre variables, y entrenamiento de un modelo de regresión. Además, se reflexionará críticamente sobre los límites del modelo, incluyendo la posible falta de datos recientes o la sobrerrepresentación de ciertos géneros o plataformas, aspectos fundamentales para garantizar la validez de la herramienta desarrollada.

Obtención y tratamiento de datos

He escogido estos datos basados en las analíticas a los videojuegos más exitosos:

- Ventas de videojuegos

Aunque este conjunto de datos podría no estar completamente actualizado, ya que el mercado de los videojuegos evoluciona muy rápido y constantemente se lanzan nuevos títulos que también deberían considerarse, lo he escogido porque ofrece una base sólida y representativa con la que trabajar. Considero que incluye una selección adecuada de videojuegos relevantes, lo que permite realizar un análisis significativo y útil para los objetivos del proyecto.

El conjunto de datos ofrece una visión completa de las ventas de videojuegos a nivel global, abarcando información clave como el título del juego, la plataforma, el año de lanzamiento, el género, la empresa distribuidora y las cifras de ventas en distintas regiones (Norteamérica, Europa, Japón, Resto del Mundo y globales).

	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.01	3.28	2.96	33.00
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	1.00	31.37

Como se puede observar ya los primeros 5 juegos son de los juegos con más éxito y por lo tanto de los más conocidos a nivel mundial, como el *Pokemon* o el *Mario Kart Wii*.

Cuenta con 16598 datos y 11 columnas (las nombradas anteriormente).

```
videojuegos.shape
```

```
(16598, 11)
```

Para limpiar estos datos he decidido mirar si había algún valor nulo para eliminarlo y que no cuenta para el estudio así nos ahorramos posibles errores a la hora de la predicción, ya que recordemos que un mínimo error puede causar la pérdida de millones de euros.

```
[156] videojuegos.dropna(inplace = True)
```

```
videojuegos.isnull().sum()
```

	0
Rank	0
Name	0
Platform	0
Year	271
Genre	0
Publisher	58
NA_Sales	0
EU_Sales	0
JP_Sales	0
Other_Sales	0
Global_Sales	0

dtype: int64

```
videojuegos.isnull().sum()
```

	0
Rank	0
Name	0
Platform	0
Year	0
Genre	0
Publisher	0
NA_Sales	0
EU_Sales	0
JP_Sales	0
Other_Sales	0
Global_Sales	0

dtype: int64

Muestra de datos:

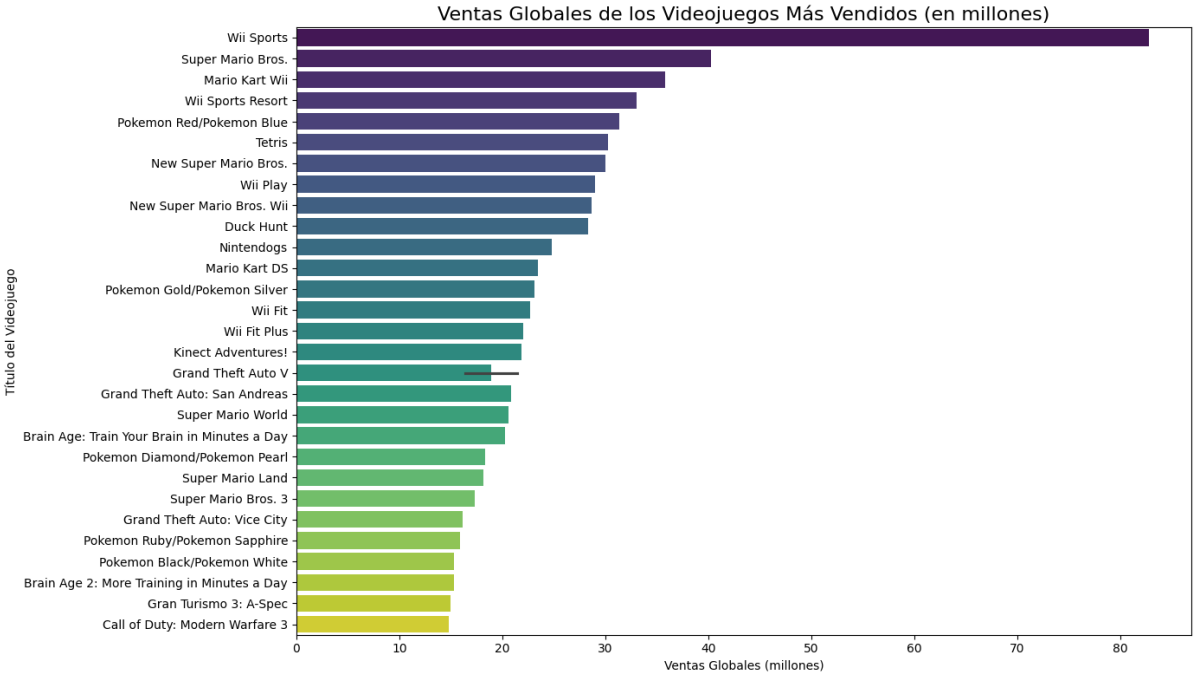
Considero que una representación gráfica de los datos es más adecuada, especialmente cuando hablamos de ventas y puntuaciones de crítica. Utilizar gráficos facilita una visualización clara y permite realizar un análisis general más efectivo, ya que proporciona una perspectiva rápida sobre las relaciones y patrones en los datos, haciendo que la información sea más accesible y comprensible.

Utilizando las librerias de *matplotlib* y *seaborn* he creado unos gráficos que ayudan a ver la relación entre los datos.

Uno de ellos es un gráfico de barras que muestra los 30 juegos principales según los millones generados en ventas. Este tipo de gráfico permite observar de manera clara y rápida cuáles son los títulos más exitosos en términos de ingresos, proporcionando una perspectiva inmediata sobre las tendencias y diferencias entre los juegos más vendidos.

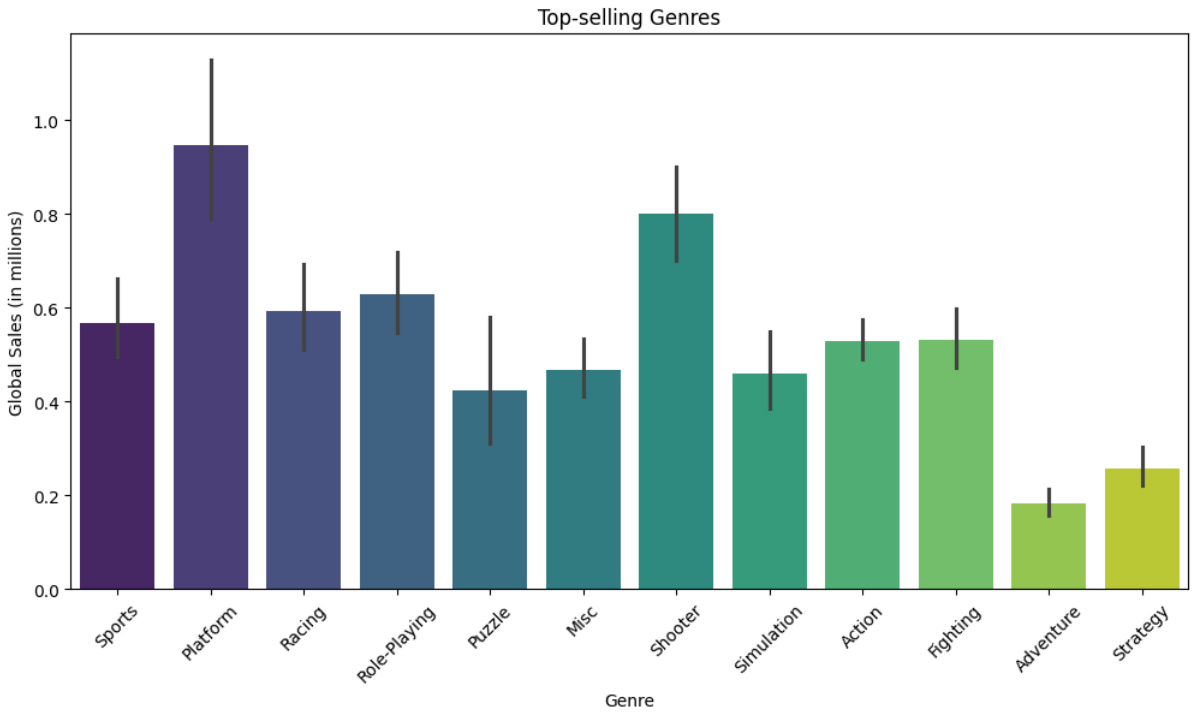
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(14, 8))
sns.barplot(x=data['Global_Sales'].head(30), y=data["Name"].head(30), palette="viridis")
plt.title("Ventas Globales de los Videojuegos Más Vendidos (en millones)", fontsize=16)
plt.xlabel("Ventas Globales (millones)")
plt.ylabel("Título del Videojuego")
plt.tight_layout()
plt.show()
```



Por último, he incluido un gráfico que muestra las ventas globales promedio por género. Este gráfico permite observar fácilmente qué género de videojuegos ha generado mayores ventas a nivel global, así como cuál tiene las ventas más bajas. De esta manera, se puede identificar rápidamente qué tipos de juegos están dominando el mercado y cuáles tienen un desempeño más modesto.

```
plt.figure(figsize=(12, 6))
sns.barplot(x='Genre', y='Global_Sales', data=data, palette='viridis')
plt.title('Top-selling Genres')
plt.xlabel('Genre')
plt.ylabel('Global Sales (in millions)')
plt.xticks(rotation=45)
plt.show()
```



Se puede observar que los videojuegos con mayores ventas suelen pertenecer a géneros más accesibles y populares, como los juegos de plataformas, destacando títulos como *Super Mario Bros*. En cambio, los juegos de aventura y estrategia, como *Plantas vs. Zombies*, tienden a tener un menor volumen de ventas.

Una explicación es que estos juegos requieren mayor concentración, resolución de acertijos o planificación estratégica, lo que puede resultar menos atractivo para un público más amplio que busque experiencias más relajadas o inmediatas. Muchos jugadores prefieren títulos que les permitan desconectar o divertirse sin una alta carga cognitiva, lo que limita la popularidad comercial de géneros como la aventura o la estrategia, a pesar de su valor narrativo o creativo.

Desarrollo de la solución IA/Big Data

Random Forest Regressor

He escogido como modelo Random Forest por varias razones como la capacidad de captura de relaciones complejas entre las variables (no lineales), muy preciso y robusto al ruido de los datos, maneja bien las variables numéricas y categóricas (una vez codificadas), y funciona bien con los datos estructurados como el que se está utilizando.

Dado que el objetivo es predecir las ventas en Europa, el modelo debe tener en cuenta múltiples factores como la plataforma, las ventas en otras regiones, el género del videojuego, el año de lanzamiento y la consola. Estas variables influyen de forma

significativa en el comportamiento de los consumidores europeos, por lo que un modelo flexible y robusto como Random Forest es una buena elección.

Importamos las librerías necesarias:

- Numpy: Fundamental para trabajar con vectores, matrices y operaciones numéricas.
- Pandas: Permite la carga, explora y la manipulación de los datos fácilmente
- Sklearn: Libreria de machine learning muy completa, con las siguientes funciones:
  - Train_test: Divide los datos en un conjunto de entrenamiento y prueba
  - Mean_squared_error: Promedio del error cuadrado entre lo real y lo predicho
  - R2_score: Mide qué tan bien el modelo explica la variación en los datos
- RandomForestRegressor: Modelo de aprendizaje automático basado en muchos árboles de decisión (problemas de regresión)
- LabelEncoder: Convierte columnas de texto en números

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import LabelEncoder
```

Luego de realizar la limpieza de datos eliminando los valores nulos, procedí a convertir las variables categóricas (*Platform*, *Genre* y *Publisher*) en variables numéricas utilizando la clase LabelEncoder.

Para ello, creé una instancia de LabelEncoder para cada una de estas columnas, ya que cada una contiene categorías distintas. Esta conversión fue necesaria para que el modelo de regresión pudiera interpretar correctamente estos datos de entrada para poder realizar la predicción.

```
le_platform = LabelEncoder()
le_genre = LabelEncoder()
le_publisher = LabelEncoder()

videojuegos['Platform'] = le_platform.fit_transform(videojuegos['Platform'])
videojuegos['Genre'] = le_genre.fit_transform(videojuegos['Genre'])
videojuegos['Publisher'] = le_publisher.fit_transform(videojuegos['Publisher'])
```

Ahora partiremos los datos de las variables predictoras (*Plataform*, *Year*, *Genre*, *Publisher*, *Na_Sales*, *Jp_Sales*, *Other_Sales*).

Por otro lado, tenemos la variable objetivo, que es lo que el modelo debe predecir las ventas globales *(EU_Sales)*

```
[160] X = videojuegos[['Platform', 'Year', 'Genre', 'Publisher', 'NA_Sales', 'JP_Sales', 'Other_Sales']]
      y = videojuegos['EU_Sales']
```

La razón por la que dividimos los datos es para realizar el paso esencial de separar los datos en conjuntos de entrenamiento y prueba: el 80% se usa para entrenar el modelo y el 20% restante se reserva para evaluar su rendimiento.

```
[12] X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Se crea una instancia del modelo con 100 árboles en el bosque. El parámetro *random_state* asegura la reproducibilidad de los resultados

```
[13] model = RandomForestRegressor(n_estimators=100, random_state=42)
```

Se entrena el modelo utilizando el conjunto de entrenamiento

```
model.fit(X_train, y_train)
```

Se realizan predicciones en base al conjunto de prueba utilizado el modelo anteriormente entrenado.

```
[15] y_pred = model.predict(X_test)
```

Se calcula el error cuadrático medio (MSE), que mide el promedio de los errores al cuadrado entre los valores predichos y reales. Un MSE de 0 significa predicciones perfectas, y cuanto menor sea mejor es el rendimiento del modelo.

El coeficiente de Determinación (R2) indica la proporción de la varianza en la variable dependiente. Un R2 de 1 significa que el modelo explica perfectamente la variabilidad de los datos, de 0 indica que el modelo no explica ninguna variabilidad, y valores negativos es peor que el modelo use la media de los datos como predicción.

```
[16] mse = mean_squared_error(y_test, y_pred)
     r2 = r2_score(y_test, y_pred)

     # Mostrar los resultados
     print(f"Error cuadrático medio (MSE): {mse}")
     print(f"Coeficiente de determinación (R²): {r2}")
```

Como se puede ver las dos dan valores que hacen saber que el modelo funciona correctamente, aunque podría ser mejorable. (MSE: Número cercano al 0, R2: Número casi aproximado a 1)

```
Error cuadrático medio (MSE): 0.1495491278272172
Coeficiente de determinación (R²): 0.7085667398024575
```

Se ha realizado una predicción con un ejemplo hipotético de un videojuego que será lanzado en 2025 para la plataforma PC, del género acción, publicado por Capcom. Según los valores estimados de ventas en otras regiones:

- América del Norte: 10 millones
- Japón: 2.5 millones
- Otras regiones: 5.3 millones

El modelo predice unas ventas en Europa de aproximadamente 8.89 millones, lo que sugiere un potencial comercial relevante en esa región. Esta información puede ser útil para tomar decisiones estratégicas como:

- Traducir y localizar el contenido al mercado europeo (subtítulos, doblaje).
- Determinar si se justifica la distribución física.
- Planificar campañas de marketing con influencers y medios regionales.

```
nuevo_juego = {
    'Platform': 'PC',
    'Year': 2025,
    'Genre': 'Action',
    'Publisher': 'Capcom',
    'NA_Sales': 10.0,
    'JP_Sales': 2.5,
    'Other_Sales': 5.3
}

nuevo_df = pd.DataFrame([nuevo_juego])

nuevo_df['Platform'] = le_platform.transform(nuevo_df['Platform'])
nuevo_df['Genre'] = le_genre.transform(nuevo_df['Genre'])
nuevo_df['Publisher'] = le_publisher.transform(nuevo_df['Publisher'])

prediccion = model.predict(nuevo_df)
print(f"Predicción de ventas en Europa: {prediccion[0]:.2f} millones")
```

```
Predicción de ventas en Europa: 8.89 millones
```

Visualización del modelo por StreamLit:

- Proyecto Random Forest Regressor

Redes Neuronales

Para realizar una comparación con el modelo anterior, he optado por utilizar el modelo de redes neuronales.

En primer lugar, he aplicado el mismo preprocesamiento de datos que utilicé con el modelo Random Forest Regressor, con el fin de asegurar una comparación más coherente y justa. Por tanto, estos primeros pasos ya se han explicado y no presentan novedades.

```
[55] videojuegos = pd.read_csv("Video_Games_Sales.csv")
     videojuegos.dropna(inplace=True)
```

```
[56] le_platform = LabelEncoder()
     le_genre = LabelEncoder()
     le_publisher = LabelEncoder()

     videojuegos['Platform'] = le_platform.fit_transform(videojuegos['Platform'])
     videojuegos['Genre'] = le_genre.fit_transform(videojuegos['Genre'])
     videojuegos['Publisher'] = le_publisher.fit_transform(videojuegos['Publisher'])
```

```
[57] X = videojuegos[['Platform', 'Year', 'Genre', 'Publisher', 'NA_Sales', 'JP_Sales', 'Other_Sales']]
     y = videojuegos['EU_Sales']
```

A este paso añadimos que estandarizamos los datos para normalizar las características numéricas, lo que ayuda a que el modelo de redes neuronales converja más rápido y de manera más estable durante el entrenamiento. Esta normalización es fundamental, ya que las redes neuronales son sensibles a las escalas de las variables de entrada.

```
[58] scaler = StandardScaler()
     X_scaled = scaler.fit_transform(X)
```

Después simplemente dividimos los datos en 80% entrenamiento y 20% de prueba.

```
[59] X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

Para este modelo añadimos la librería de TensorFlow, la cual proporciona las herramientas necesarias para construir, entrenar y evaluar redes neuronales de manera eficiente y flexible. Gracias a TensorFlow, podemos definir arquitecturas complejas, optimizar los parámetros del modelo y aprovechar aceleración por hardware como GPUs para mejorar el rendimiento del entrenamiento.

```
[60] import tensorflow as tf
     from tensorflow import keras
```

Este código define un modelo de red neuronal secuencial con tres capas densas (fully connected). La primera capa tiene 64 neuronas con activación ReLU y recibe entradas con el número de características de los datos. La segunda capa tiene 32 neuronas también con activación ReLU. Finalmente, la última capa es una única neurona sin activación, que produce la predicción continua (ventas en Europa).

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)
])
```

Se configura el modelo para entrenarlo usando el optimizador Adam, que ajusta los pesos de forma eficiente. La función de pérdida es el error cuadrático medio (MSE), ideal para problemas de regresión. Además, se monitorea el error absoluto medio (MAE) como métrica adicional.

Luego, el modelo se entrena durante 20 épocas con un tamaño de lote de 32 muestras, usando el 20% de los datos de entrenamiento para validación. El parámetro verbose=1 muestra el progreso del entrenamiento.

```
[ ] model.compile(optimizer='adam', loss='mse', metrics=['mae'])

history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2, verbose=1)

Epoch 1/20
326/326 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.1666 - mae: 0.1683 - val_loss: 0.0856 - val_mae: 0.0875
Epoch 2/20
326/326 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0385 - mae: 0.0782 - val_loss: 0.0974 - val_mae: 0.0919
Epoch 3/20
326/326 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0304 - mae: 0.0743 - val_loss: 0.0874 - val_mae: 0.0907
Epoch 4/20
326/326 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0474 - mae: 0.0804 - val_loss: 0.0620 - val_mae: 0.0755
```

Aquí podemos ver que efectivamente el modelo de redes neuronales es mucho mejor para este tipo de predicción que el anteriormente hecho:

```
y_pred_nn = model.predict(X_test).flatten()

mse_nn = mean_squared_error(y_test, y_pred_nn)
r2_nn = r2_score(y_test, y_pred_nn)

print(f"Error cuadrático medio (MSE) - Red Neuronal: {mse_nn}")
print(f"Coeficiente de determinación (R²) - Red Neuronal: {r2_nn}")

102/102 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
Error cuadrático medio (MSE) - Red Neuronal: 0.051583453760352976
Coeficiente de determinación (R²) - Red Neuronal: 0.8994769523564357
```

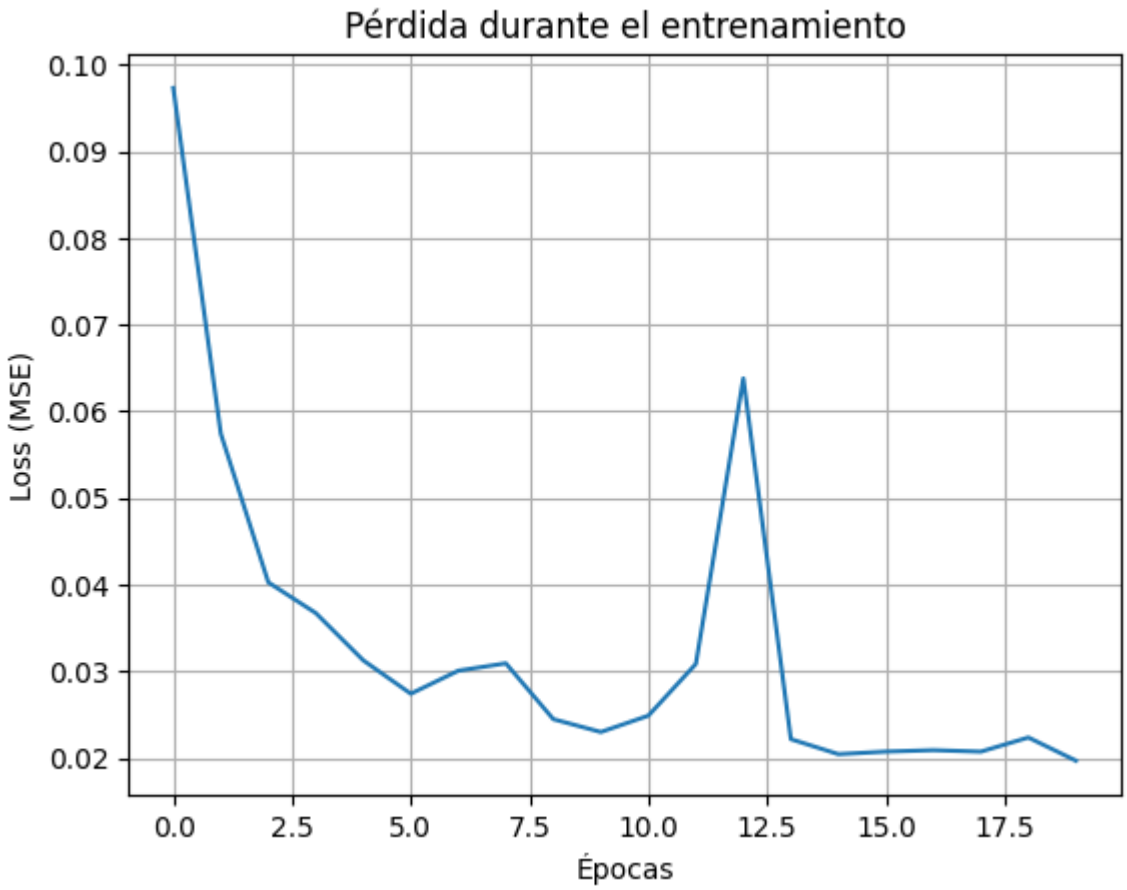
Observamos que el MSE es muy bajo, cercano a cero, lo que indica que el modelo comete pocos errores en sus predicciones. Además, el coeficiente de determinación (R^2) está próximo a uno, lo que significa que el modelo explica la mayor parte de la variabilidad de los datos y tiene un buen ajuste. Por lo tanto, ambos indicadores reflejan un desempeño excelente del modelo.

Se genero un gráfico que muestra cómo evoluciona la función de pérdida (*loss*) durante el entrenamiento del modelo a lo largo de las épocas, lo que ayuda a evaluar el proceso de aprendizaje del modelo durante el entrenamiento.

- El eje X representa las épocas
- El eje Y muestra el valor de la pérdida (MSE) en cada época.

```
import matplotlib.pyplot as plt

plt.plot(history.history['loss'])
plt.title('Pérdida durante el entrenamiento')
plt.xlabel('Épocas')
plt.ylabel('Loss (MSE)')
plt.grid(True)
plt.show()
```



La curva muestra cómo la pérdida disminuye rápidamente en las primeras épocas, lo que indica que el modelo está aprendiendo y mejorando su capacidad para ajustar los datos. Luego, alrededor de la época 12, hay un pequeño aumento en la pérdida, seguido de una caída brusca, lo cual podría indicar una ligera fluctuación en el proceso de optimización, pero luego la pérdida vuelve a estabilizarse en un valor bajo cercano a 0.02.

Por último, se implementó un código que permite realizar predicciones de las ventas estimadas en Europa, basándose en las características específicas del videojuego.

```
nuevo_juego = {
    'Platform': 'PC',
    'Year': 2025,
    'Genre': 'Action',
    'Publisher': 'Capcom',
    'NA_Sales': 15.0,
    'JP_Sales': 2.5,
    'Other_Sales': 10.3
}

nuevo_df = pd.DataFrame([nuevo_juego])
nuevo_df['Platform'] = le_platform.transform(nuevo_df['Platform'])
nuevo_df['Genre'] = le_genre.transform(nuevo_df['Genre'])
nuevo_df['Publisher'] = le_publisher.transform(nuevo_df['Publisher'])

nuevo_df_scaled = scaler.transform(nuevo_df)

prediccion_nn = model.predict(nuevo_df_scaled)
print(f"Predicción de ventas en Europa (red neuronal): {prediccion_nn[0][0]:.2f} millones")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
Predicción de ventas en Europa (red neuronal): 18.29 millones
```

Visualización del modelo por StreamLit:

- Proyecto Redes Neuronales

Comparativa

Habiendo realizado dos modelos, toca el momento de comparar los resultados y entender porque uno es mejor que el otro. Primero interpretando las métricas, recordando que el MSE (Mean Squared Error) mide el promedio del cuadrado de los errores entre las predicciones del modelo y los valores reales, siendo que si da un valor bajo indica buena precisión, mientras que el R2 (R-squared) indica que también se ajusta el modelo a los datos reales, sabiendo que cuanto más cerca de 1 mejor.

Sabiendo esto podemos hacer el estudio de las métricas dadas en los dos modelos:

MÉTRICA	RANDOM FOREST	RED NEURONAL
MSE	0.15	0.015
R2	0.70	0.89

Se puede observar bien que la red neuronal supera al Random Forest en ambas métricas. Por un lado, el MSE es diez veces menor lo que indica que las predicciones son mucho más cercanas a los valores reales. Por otro lado, el R2 de la Red neuronal explica que el 89% de la varianza en las ventas europeas, por lo tanto, se ve mucho mejor que el 70% de la varianza del otro modelo.

¿Porque entonces la red neuronal es el modelo para este caso?

Por la capacidad que tiene para modelar relaciones no lineales complejas, ya que la relación entre las ventas de distintas regiones y las características del juego es no lineal y multifactorial, lo que indica que hay muchos valores que influyen en el resultado .
Tiene un mejor ajuste al problema específico, en cambio Random Forest puede no captar todas las complejidades de interacciones entre las variables.

Conclusión

Tras la evaluación de los dos modelos y la prueba de ellos se concluye que la mejor manera de estimar las ventas de los videojuegos en Europa es a través del modelo de Red neuronal que ofrece un mejor desempeño, con menor error (MSE) y mayor capacidad explicativa (R2).

Sim embargo, es importante destacar que el rendimiento del modelo no solo depende del algoritmo elegido sino de la calidad de los datos y la cantidad de información que tenemos, con esto quiero decir que estamos recogiendo datos como el género del juego, la plataforma, el año de salida, la empresa y las ventas regionales, pero se debería tener en cuenta muchas más variables que no están dadas como las puntuaciones dadas, tanto por los expertos como por el público que a lo mejor ha tenido acceso a versiones anteriores a la definitiva (una beta por ejemplo), la competencia de mercado más actual, ya que estos juegos aunque es cierto que son bastante conocidos y populares y es importante para saber cuáles tienen éxito y cuales no, tendríamos que ir actualizando esta lista a medida que salen juegos que tienen cierto impacto y tenerlos en cuenta para el estudio y la evolución del mundo de los videojuegos. También el precio de los juegos es importante, ya que actualmente se puede observar cómo los precios aumentan y es algo para tener en cuenta para el estudio del mercado, al igual que la competencia, ya no solo de juegos que se han actualmente publicado sino también de los que vienen a futuro, ya que publicar un juego que puede ser competencia directa con otro y que salgan en el mismo momento o cercano puede también afectar a las ventas por las comparativas que pueda hacer el público. Las tendencias regionales o culturales también pueden afectar a las ventas por ello en las diferentes regiones se vendan más o menos.

Con todo esto nos podemos dar cuenta de la cantidad de variables que ni siquiera son posibles añadir a un dataframe pero que es importante tener en cuenta, por tanto, este modelo creado está bien para una estimación rápida, pero es importante hacer un estudio más profundizado de las ventas y saber si puede salir rentable o no.

Código completo en GitHub:

- Proyecto_modelo